

Partial Assembly by Projection: an inexact matrix-free apply for the SFEM code generator

Abstract

The matrix-free apply of a nonlinear elasticity operator carries a quadrature loop because the material tangent varies across each element. Projecting that tangent onto the constants separates the loop into a small per-element object and a tensor that belongs to the reference element alone. The reference tensor can then be integrated symbolically once, at generation time, and it turns out to be far smaller than its size suggests: it is a Gram matrix, so its rank is the dimension of the span of the element's gradient functions. For a linear simplex that rank is one, and the construction reduces exactly to the loperand. This note states the technique, the compression, what the code generator emits, how it vectorises, and the measurements taken so far.

1 The exact apply, and why it has a loop

Let K be an element with reference cell $\widehat{K}$ and affine or isoparametric map $\mathbf{x}(\boldsymbol{\xi})$, Jacobian J , and $dV = \det J d\boldsymbol{\xi}$. For a hyperelastic energy with first Piola stress $P(F)$, the matrix-free action of the second form on an increment h is

$$(\mathcal{H}h)_{i,p} = \int_K S_{ijkl}(F(\mathbf{x})) \frac{\partial h_k}{\partial x_l} \frac{\partial \varphi_p}{\partial x_j} dV, \quad S_{ijkl} = \frac{\partial P_{ij}}{\partial F_{kl}}, \quad (1)$$

with i the output component, p the test function, and φ_p the basis.

Pull the derivatives back to the reference cell with $\partial/\partial x_j = \sum_n J_{nj}^{-1} \partial/\partial \xi_n$ and collect the geometry into the tangent:

$$\overline{S}_{ikmn} = \sum_{j,l} S_{ijkl} J_{nj}^{-1} J_{ml}^{-1} \det J, \quad (2)$$

so that (1) becomes

$$(\mathcal{H}h)_{i,p} = \int_{\widehat{K}} \overline{S}_{ikmn}(\boldsymbol{\xi}) \frac{\partial h_k}{\partial \xi_m} \frac{\partial \varphi_p}{\partial \xi_n} d\boldsymbol{\xi}. \quad (3)$$

This $\overline{S}$ is the *loperand*: the flux differentiated once and already contracted with the geometry, so that the element vector is nothing but differences of its contractions. It is the same object SFEM's hand-written kernels build, and the same one `plans/loperand.py` isolates from a material's flux.

Everything under the integral except $\overline{S}$ is a property of the reference element. What forces a quadrature loop is that $\overline{S}$ depends on $\boldsymbol{\xi}$, because F does.

2 The projection

Replace $\overline{S}$ by its L^2 projection onto the constants on K , that is by its element average $\overline{S}^0 = |\widehat{K}|^{-1} \int_{\widehat{K}} \overline{S}$. It leaves the integral and the sum separates:

$$(\mathcal{H}h)_{i,p} \approx \overline{S}_{ikmn}^0 \sum_j h_{k,j} \overline{W}_{jmpn}, \quad \overline{W}_{jmpn} = \int_{\widehat{K}} \frac{\partial \varphi_j}{\partial \xi_m} \frac{\partial \varphi_p}{\partial \xi_n} d\boldsymbol{\xi}. \quad (4)$$

$\bar{S}^0$ is one small object per element, computed once from the state. $\bar{W}$ belongs to the element and is integrated exactly, symbolically, at generation time. *The quadrature loop leaves the kernel entirely.*

Where the approximation is, and where there is none

On an affine simplex the deformation gradient is constant over the cell, so $\bar{S}$ is constant, so projecting it onto the constants returns it unchanged and (4) is an identity rather than an approximation. This holds for a nonlinear material as much as a linear one: it is a statement about the geometry, not about P .

That is the load-bearing fact of the whole construction, because it gives an exact reference to test against. On TRI3 and TET4 the projected kernel must reproduce the exact one; only on curved or higher-order elements is there an error, and there it must be measured rather than assumed.

Symmetry

The element matrix entry implied by (4) is $\bar{S}_{ikmn}^0 \bar{W}_{jpmn}$. Requiring it to be symmetric under exchanging the two (component, node) pairs, and using $\bar{W}_{jpmn} = \bar{W}_{pnjm}$, forces

$$\bar{S}_{ikmn}^0 = \bar{S}_{kinm}^0. \quad (5)$$

The pair that exchanges is (i, n) against (k, m) : the output component with the *test* direction, and the increment component with the *increment* direction. Packing on (i, k) against (m, n) instead is the plausible-looking error, and it is invisible to any test that compares the construction against itself. Under (5), $\bar{S}^0$ has 45 independent components in three dimensions and 10 in two.

3 Compression of the reference tensor

3.1 What it contains

$\bar{W}$ is integrated exactly, so its entries are rationals. There are very few of them, and on the simplices most are zero.

element	entries	distinct values	zeros	projection
TRI3	36	3	20	exact
TET4	144	3	108	exact
QUAD4	64	6	0	inexact
HEX8	576	10	0	inexact
TET10	900	9	519	inexact

Table 1: The symbolically integrated reference tensor. At most ten distinct rationals for any element.

3.2 Rank, which is where the structure really is

The zeros are a symptom. $\bar{W}$, read as a matrix on the pair index (j, m) , is the Gram matrix of the element's gradient functions $\{\partial\varphi_j/\partial\xi_m\}$ in $L^2(\hat{K})$. Its rank is therefore the dimension of the span of those functions and nothing more.

A linear simplex is rank one because its gradients are constants. Rank one is precisely the loperand of Section 1: one object per element, contracted once. So the rank factorisation is not

element	order	rank	span of the gradient functions
TRI3	6	1	constants
TET4	12	1	constants
QUAD4	8	3	1, ξ_0 , ξ_1
HEX8	24	7	1 and the six bilinear monomials
TET10	30	4	1, ξ_0 , ξ_1 , ξ_2

Table 2: Rank of the reference tensor, which equals the dimension of the span of the element’s gradient functions exactly.

a second idea beside the loperand; it is the same structure carried to elements whose gradients are not constant.

Factor exactly over the rationals,

$$\overline{W} = C^\top X C, \quad C \in \mathbb{Q}^{r \times Nd}, \quad X \in \mathbb{Q}^{r \times r}, \quad (6)$$

with r the rank, N the node count and d the dimension. The factorisation is checked against the tensor it factors when it is built.

4 The generated kernel

4.1 Staging

Substituting the factorisation into (4) and naming each intermediate gives four stages:

$$P_{kam} = \sum_j C_{a,(j,m)} h_{k,j} \quad \text{the increment, compressed} \quad (7)$$

$$Y_{ian} = \sum_{k,m} \overline{S}_{ikmn}^0 P_{kam} \quad \text{the tangent applied} \quad (8)$$

$$Q_{ibn} = \sum_a X_{ab} Y_{ian} \quad \text{the middle factor} \quad (9)$$

$$\text{out}_{ip} = \sum_{b,n} C_{b,(p,n)} Q_{ibn} \quad \text{expanded back to nodes} \quad (10)$$

Every sum skips its zero coefficients, so a zero of C or X never enters an expression rather than being cancelled out of one later.

The staging is the point and not an implementation detail. Expanding the four stages into one expression per output and letting common-subexpression elimination rediscover the structure returns the dense operation count exactly; the saving lives in keeping the intermediates named.

4.2 Fold the constants, do not tabulate them

The entries of $\overline{W}$ are known at generation time, so they are emitted as literals. The tempting alternative — store the distinct values in a runtime table and index it — compresses the memory and destroys the arithmetic: the constants stop being constants, the compiler can no longer fold them, and every multiply by a zero entry survives into the emitted code.

This is not hypothetical. The existing prototype (`python/codegen/neohookean_partial_assembly.py`) emits, for TET4, a table holding exactly 1, -1 and 0, and its kernels then multiply by the zero entry twelve times each; the reported “97.9% memory saving” buys 141 stored numbers that were 0 and ± 1 at a cost of several hundred floating-point operations.

element	rank/order	dense, folded	staged, folded	runtime table
TRI3	1/6	96	52	244
TET4	1/12	486	207	1467
QUAD4	3/8	238	238	431
HEX8	7/24	4059	2610	6330
TET10	4/30	4018	1380	8349

Table 3: Operations in the element apply, counted after common-subexpression elimination. Preliminary: these are symbolic operation counts, not measured run times.

Table 3 says where the staging pays. The saving tracks the rank against the order: TET10 at 4/30 gains $2.9\times$ and TET4 at 1/12 gains $2.4\times$, while QUAD4 at 3/8 comes out exactly even — not enough structure to beat common-subexpression elimination on the dense form. Against the runtime-table form the improvement is $7.1\times$ on TET4 and $6.1\times$ on TET10.

5 Vectorisation

The generated kernels process a block of `VECTOR.SIZE` elements at a time, with the lane loop innermost and marked `#pragma omp simd`. The staged form suits that layout well, and for reasons worth stating separately.

The reference tensor is lane-invariant. C and X are compile-time rationals, identical for every element and therefore for every lane. Folded, they appear as immediates: no load, no broadcast, and a multiply by ± 1 or by a power of two is folded away entirely. Behind a runtime table the same values become a memory reference whose address does not depend on the lane — a broadcast load rather than an immediate — and, worse, an opaque value that no longer permits folding. This is the arithmetic half of the cost in Table 3.

The tangent is a per-element stream. $\bar{S}^0$ is 45 numbers per element in three dimensions, stored one component-array per index in the structure-of-arrays layout the framework already uses for geometry. Each component is then a unit-stride read across the lane loop, which vectorises without gather.

The intermediates are lane-width vectors. P , Y and Q are per-element scalars, so in a blocked kernel each becomes one vector register. Their counts are the stage sizes in Table 4, and they are the binding constraint: on HEX8, $63 + 63 + 63$ live intermediates at a lane width of eight doubles will not stay in registers, and the kernel will spill. Rank is what controls this, since all three stage sizes are $d^2 r$.

The output scatter is unchanged. The last stage writes one value per node per component, exactly as the existing apply kernels do, so it reuses the same scatter — atomic on the plain traversal, and plain accumulation into thread-private scratch on the packed one.

A consequence worth drawing out: on the elements where the projection is exact the register pressure is also lowest, and on HEX8 — where the projection is an approximation whose error is not yet measured — it is highest. The two difficulties arrive together, and a rank-truncated factorisation (keeping $r' < r$ modes) is the natural knob for both, at the price of a second and larger approximation.

element	rank	$ P $	$ Y $	$ Q $	outputs
TRI3	1	4	4	4	6
TET4	1	9	9	9	12
QUAD4	3	12	12	12	8
HEX8	7	63	63	63	24
TET10	4	36	36	36	30

Table 4: Live intermediates per element in each stage. In a blocked kernel each is one vector register, so these are the register-pressure figures.

6 Verification

The construction is checked at three independent points, since each stage can be wrong in a way the next cannot see.

1. **The symbolic integration**, against four-point Gauss–Legendre quadrature on QUAD4 and HEX8, entry by entry: agreement to 10^{-16} .
2. **The factorisation**, against the tensor it factors: $C^\top X C = \overline{W}$ exactly, over the rationals.
3. **The action itself**, against (1) integrated directly on a concrete tetrahedron, for linear elasticity and for neohookean Ogden: agreement is exact, a rational zero rather than round-off, for the dense and the staged form alike. The second material also demonstrates the claim of Section 2, that on an affine simplex the projection is lossless even when the material is nonlinear.

Only the third of these can catch a wrong index convention, and it did: the symmetry (5) had been implemented as (i, k) against (m, n) , which the first two checks and a comparison of the staged form against the dense form all pass, because they share the error.

7 Status

Implemented in the code generator’s planning layer as `plans/inexact_apply.py`, with the reference bases in `fem/reference_basis.py`. The plan is consulted by nothing: generated output is byte-identical, and generation of the kernel is intended to be opt-in when it is wired.

Not yet done, in order:

1. the opt-in flag, threaded from the material through the pipeline;
2. a kernel computing $\overline{S}^0$ per element from the state — for an affine simplex this is (2) evaluated once, and for a curved element it is the average, which is where the projection is actually taken;
3. the staged apply as its own entry point;
4. a gate that compares against the exact apply: bit-parity on TRI3 and TET4, where the projection is exact, and a *measured relative error* on HEX8 and TET10, where a digest comparison is meaningless because the kernel is deliberately a different operator.

The numbers in Tables 3 and 4 are symbolic operation and register counts, not timings.

material	element	ndof	exact	projected	ratio	rel. difference
linear elasticity	TET4	46875	18.65	16.61	$0.89\times$	1.5e-16
linear elasticity	TET4	206763	17.79	15.24	$0.86\times$	1.7e-16
linear elasticity	HEX8	46875	26.11	17.01	$0.65\times$	4.8e-15
linear elasticity	HEX8	206763	23.24	14.98	$0.64\times$	1.1e-14
neohookean Ogden	TET4	46875	5.26	4.83	$0.92\times$	1.8e-15
neohookean Ogden	TET4	206763	4.91	4.44	$0.90\times$	3.8e-15
neohookean Ogden	HEX8	46875	7.60	3.41	$0.45\times$	1.0e-04
neohookean Ogden	HEX8	206763	6.97	3.27	$0.47\times$	3.6e-05

Table 5: MDOF/s for the exact and projected apply. The projection is exact wherever the tangent does not vary over the element, and the only genuinely inexact row – neohookean on HEX8 – converges under refinement, from 8.9e-4 to 3.6e-5 across five levels.

Measured throughput, and what it refutes

With the opt-in wiring in place the kernel can be timed. Single-threaded, `-O3 -march=native`, best of five, at and above the saturation point of roughly 47k degrees of freedom:

Two claims of Section 2 hold: the projection is exact at round-off wherever the tangent is constant over the element, including on HEX8 where the kernel really does average over eight points; and where it is an approximation, it converges.

The claim of Section 4 does not. **The projected apply is slower everywhere**, from $0.90\times$ on TET4 to $0.45\times$ on HEX8, and worst exactly where it was supposed to pay.

Table 3 is not wrong, but it counts only the contraction. It does not count forming the tangent, and the projected kernel must *materialise* $\bar{S}$ — forty-five independent components — at every quadrature point in order to average it. The exact kernel never forms the tangent at all: it computes the directional linearisation $P'(F)[\nabla h]$ at each point, a vector-sized quantity rather than a rank-four one. On HEX8 that trades eight cheap linearisations for eight full tangent evaluations, and the saved contraction does not come close to paying for it.

So the fused kernel — the one wired here, which rebuilds the tangent on every apply — cannot win, and tuning the contraction will not change that.

Where the technique can still pay

Partial assembly pays when the tangent is formed once and *reused*. In a Newton step the tangent is fixed while the Krylov solve applies the operator tens or hundreds of times. Writing $\bar{S}$ into storage once per Newton iteration and applying it thereafter costs, per apply, only the staged contraction — which is the quantity Table 3 does measure.

That is the measurement worth taking next, and it needs the split kernels: one writing $\bar{S}$ per element, one reading it. Its break-even is the number of applies per tangent, which the fused form fixes at one and a Krylov solve makes large. Until it is measured, the honest summary of this note is that the mathematics is verified, the compression is real, and the fused kernel is a demonstration rather than an optimisation.